

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2017 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2017	2
SUBIECTUL I (60 de puncte)	2
1. Algoritmul Dijkstra pentru drumuri de cost minim	2
2. Formatarea documentelor: Imagini și diagrame	2
3. Programare: Cifre ordonate crescător	2
4. Baze de date: Zboruri	3
II. Rezolvare Definitivat Varianta 3 (20 aprilie 2017)	3
SUBIECTUL I	4
1. Metoda Divide et Impera	4
2. Animații în prezentări electronice	4
3. Programare: Numere cu exact două cifre distincte, consecutive	4
4. Baze de date: Costume de teatru	6
SUBIECTUL al II-lea - Completare pentru Model 2017 (30 de puncte)	7
1. Strategie didactică pentru secvența A: tablouri unidimensionale	7
2. Itemi de completare	7
SUBIECTUL al II-lea - Completare pentru Varianta 3 2017 (30 de puncte)	7
1. Strategie didactică pentru secvența A: Divide et Impera	7
2. Itemi de completare pentru Divide et Impera și animații	7

I. Rezolvare Definitivat Model 2017

SUBIECTUL I (60 de puncte)

1. Algoritmul Dijkstra pentru drumuri de cost minim

- **Definire:** Se prezintă graful orientat ponderat, determinarea drumurilor minime de la un nod sursă la toate celelalte noduri.
- **Trace:** Execuția pe un graf ponderat ales cu cel puțin 7 noduri.
- **Problemă (Dijkstra):**

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = 1e9;
void dijkstra(int start, int n, const vector<vector<pair<int, int>>> &adj,
vector<int> &dist) {
    dist.assign(n + 1, INF);
    vector<bool> viz(n + 1, false);
    dist[start] = 0;
    for (int i = 1; i <= n; ++i) {
        int u = -1;
        for (int j = 1; j <= n; ++j) {
            if (!viz[j] && (u == -1 || dist[j] < dist[u])) u = j;
        }
        if (dist[u] == INF) break;
        viz[u] = true;
        for (auto &edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;
            if (dist[u] + w < dist[v]) dist[v] = dist[u] + w;
        }
    }
}
```

2. Formatarea documentelor: Imagini și diagrame

- **Tipuri:** Imagini fișier, forme vectoriale (Shapes), grafice dinamice (Charts).
- **Aliniere:** Raportul imaginii față de text (In Line with Text, Square, Tight).
- **Proprietăți:** Luminozitate, contrast, decupare (Crop), dimensiune și rotație.

3. Programare: Cifre ordonate crescător

Soluție C++:

```
#include <iostream>
#include <fstream>
using namespace std;

bool cifreCrescator(long long n) {
    if (n < 10) return true;
    int ultima = n % 10;
    n /= 10;
    while (n > 0) {
        int penultima = n % 10;
```

```

        if (penultima > ultima) return false;
        ultima = penultima;
        n /= 10;
    }
    return true;
}

int main() {
    ifstream fin("def.in");
    long long x;
    long long max_val = -1;
    while (fin >> x) {
        if (cifreCrescator(x)) {
            if (x > max_val) max_val = x;
        }
    }
    fin.close();

    ofstream fout("def.out");
    if (max_val == -1) fout << "nu exista\n";
    else fout << max_val << "\n";
    fout.close();
    return 0;
}

```

4. Baze de date: Zboruri

- **Entități:**

- ▶ CLIENT: id_client (PK), nume, prenume, adresa, telefon.
- ▶ ZBOR: id_zbor (PK), aeroport_plecare, aeroport_sosire, data_ora_plecare, data_ora_sosire.
- ▶ BILET: id_bilet (PK), id_client (FK), id_zbor (FK), loc, mod_plata, pret.

- **Normalizare:** Datele clientului, datele zborului și datele biletului sunt separate pentru a evita redundanța. Fiecare atribut non-cheie depinde de cheia tabelului său, iar relațiile se realizează prin chei străine.

- **Relații și restricții:** CLIENT 1:M BILET, ZBOR 1:M BILET. BILET conține datele tranzacției: loc, mod de plată, preț. Modelul respectă 1NF-3NF prin separarea datelor clientului, zborului și achiziției. Restricții utile: pret >= 0, data_ora_sosire > data_ora_plecare, loc unic pentru același zbor.

- **SQL exemplu:**

```

SELECT DISTINCT c.nume, c.prenume
FROM CLIENT c
JOIN BILET b ON c.id_client = b.id_client
WHERE b.mod_plata = 'numerar';

```

II. Rezolvare Definitivat Varianta 3 (20 aprilie 2017)

[Subiect PDF](file:

SUBIECTUL I

1. Metoda Divide et Impera

- **Principiu:** Împărțirea problemei inițiale în subprobleme similare de dimensiuni mai mici, rezolvarea recursivă a acestora și combinarea rezultatelor pentru a obține soluția finală.
- **Probleme:** Merge Sort, determinarea minimului/maximului dintr-un vector.

2. Animații în prezentări electronice

- **Tipuri:** Entrance (Intrare), Emphasis (Accent), Exit (Ieșire).
- **Proprietăți:** Start (On Click, With Previous, After Previous), Duration (Durată), Delay (Întârziere).

3. Programare: Numere cu exact două cifre distincte, consecutive

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <cmath>
using namespace std;

void minmax(long long n, int &minim, int &maxim) {
    minim = 9; maxim = 0;
    long long temp = n;
    if (temp == 0) { minim = 0; maxim = 0; return; }
    while (temp > 0) {
        int cif = temp % 10;
        if (cif < minim) minim = cif;
        if (cif > maxim) maxim = cif;
        temp /= 10;
    }
}

bool check(long long n) {
    bool ap[10] = {false};
    int count_distinct = 0;
    long long temp = n;
    if (temp == 0) return false;
    while (temp > 0) {
        int cif = temp % 10;
        if (!ap[cif]) { ap[cif] = true; count_distinct++; }
        temp /= 10;
    }
    if (count_distinct != 2) return false;
    int d1 = -1, d2 = -1;
    for (int i = 0; i < 10; ++i) {
        if (ap[i]) {
            if (d1 == -1) d1 = i;
            else d2 = i;
        }
    }
    return abs(d1 - d2) == 1;
}

int main() {
    ifstream fin("def.in");
```

```

    long long x;
    int count = 0;
    while (fin >> x) {
        if (check(x)) count++;
    }
    fin.close();
    cout << count << endl;
    return 0;
}

```

Soluție Pascal:

```

program CifreConsecutive;
var
    fin: text;
    x: int64;
    count: integer;

procedure minmax(n: int64; var minim, maxim: integer);
var cif: integer;
begin
    minim := 9; maxim := 0;
    if n = 0 then
    begin
        minim := 0; maxim := 0;
        exit;
    end;
    while n > 0 do
    begin
        cif := n mod 10;
        if cif < minim then minim := cif;
        if cif > maxim then maxim := cif;
        n := n div 10;
    end;
end;

function check(n: int64): boolean;
var
    ap: array[0..9] of boolean;
    count_distinct, d1, d2, i, cif: integer;
    temp: int64;
begin
    for i := 0 to 9 do ap[i] := false;
    count_distinct := 0;
    temp := n;
    if temp = 0 then
    begin
        check := false;
        exit;
    end;
    while temp > 0 do
    begin
        cif := temp mod 10;
        if not ap[cif] then
        begin
            ap[cif] := true;
            count_distinct := count_distinct + 1;

```

```

    end;
    temp := temp div 10;
end;
if count_distinct <> 2 then
begin
    check := false;
    exit;
end;
d1 := -1; d2 := -1;
for i := 0 to 9 do
begin
    if ap[i] then
    begin
        if d1 = -1 then d1 := i
        else d2 := i;
    end;
end;
check := abs(d1 - d2) = 1;
end;

begin
    assign(fin, 'def.in');
    reset(fin);
    count := 0;
    while not eof(fin) do
    begin
        read(fin, x);
        if check(x) then count := count + 1;
    end;
    close(fin);
    writeln(count);
end.

```

4. Baze de date: Costume de teatru

• Entități:

- ▶ CLIENT: id_client (PK), nume, prenume, adresa.
- ▶ COSTUM: id_costum (PK), personaj, marime, descriere.
- ▶ INCHIRIERE: id_client (FK), id_costum (FK), data_inchiriere, data_returnare.

• SQL:

```

UPDATE COSTUM
SET marime = marime - 1
WHERE personaj = 'Cleopatra';

```

- **Relații și normalizare:** CLIENT și COSTUM sunt în relație M:N prin INCHIRIERE, deoarece un client poate închiria mai multe costume, iar același costum poate fi închiriat de-a lungul timpului de clienți diferiți. INCHIRIERE trebuie să conțină chei străine valide și intervale calendaristice coerente (data_returnare >= data_inchiriere). Modelul respectă 1NF prin attribute atomice, 2NF prin separarea atributelor clientului/costumului de închiriere și 3NF prin evitarea dependențelor tranzitive.

—

SUBIECTUL al II-lea - Completare pentru Model 2017 (30 de puncte)

1. Strategie didactică pentru secvența A: tablouri unidimensionale

Metodă activ-participativă: învățarea prin descoperire dirijată.

Argumente: Elevii pot observa singuri necesitatea unui tablou atunci când trebuie memorate și prelucrate mai multe valori de același tip; metoda dezvoltă gândirea algoritmică prin trecerea de la exemple concrete la reguli generale de indexare și parcurgere.

Elemente de proiectare: activitate de învățare - determinarea maximului dintr-un vector; mijloc - calculator cu mediu de programare și fișă de lucru; formă de organizare - lucru pe perechi.

Scenariu: Profesorul propune memorarea notelor unei clase și cere elevilor să explice de ce variabilele separate sunt ineficiente. Elevii descoperă utilizarea vectorului $v[1..n]$, apoi implementează parcurgerea pentru determinarea maximului. Profesorul ghidează verificarea pe exemple și fixează rolul indicelui, al lungimii vectorului și al inițializării.

2. Itemi de completare

Caracteristici: răspuns scurt; spațiu liber integrat într-un enunț; verifică noțiuni precise; se corectează obiectiv; nu trebuie să permită mai multe răspunsuri ambigue.

Reguli: formulare clară; un singur spațiu important per cerință; răspunsul așteptat trebuie să fie unic sau foarte bine delimitat.

- **Secvența A:** Într-un tablou unidimensional, elementul aflat pe poziția i se accesează prin notația [spațiu liber]. **Răspuns:** $v[i]$.
- **Secvența B:** În aplicația Paint, instrumentul folosit pentru umplerea unei zone cu o culoare se numește [spațiu liber]. **Răspuns:** Umplere cu culoare / Fill.

SUBIECTUL al II-lea - Completare pentru Varianta 3 2017 (30 de puncte)

1. Strategie didactică pentru secvența A: Divide et Impera

Metodă: problematizarea. Elevii primesc sarcina de a determina maximul dintr-un vector foarte mare și sunt ghidați să observe că problema poate fi împărțită în două subprobleme similare.

Activitate: determinarea maximului prin împărțirea recursivă a intervalului. **Mijloc:** mediu de programare și schemă pe tablă a arborelui apelurilor. **Formă:** frontal pentru formularea ideii, apoi individual.

Scenariu: Profesorul pornește de la soluția iterativă, cere o alternativă recursivă, desenează împărțirea intervalului $[st, dr]$, iar elevii implementează funcția $maxim(st, dr)$. Elevii testează cazurile de bază și explică etapa de combinare prin $max(max_stanga, max_dreapta)$.

2. Itemi de completare pentru Divide et Impera și animații

- **Caracteristici și reguli:** itemul cere completarea unei informații lipsă, are răspuns scurt, se corectează obiectiv; spațiul liber nu se plasează la începutul enunțului, nu se cer termeni irelevanți, iar contextul trebuie să indice clar răspunsul.
- **Item A:** Metoda Divide et Impera presupune împărțirea problemei în subprobleme de același tip, rezolvarea lor și [spațiu liber] rezultatelor. **Răspuns:** combinarea.
- **Item B:** Într-o prezentare, un efect care stabilește apariția unui obiect pe diapozitiv este un efect de [spațiu liber]. **Răspuns:** intrare.